

EXPT NO: 3	IMPLEMENTATION OF CONVOLUTIONAL NEURAL NETWORKS (CNNs) FOR IMAGE CLASSIFICATION
DATE: 10/08/2026	

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is a Convolutional Neural Network (CNN)?

A **Convolutional Neural Network (CNN)** is a type of deep learning neural network mainly designed for processing images and visual data. It automatically learns important features such as edges, textures, shapes, and objects directly from images. A CNN generally consists of convolution layers, activation functions, pooling layers, and fully connected layers. The convolution layers extract features, while pooling layers reduce their dimensions. Finally, fully connected layers perform classification. CNNs are widely used in image classification, object detection, face recognition, medical image analysis, and autonomous driving.

2. What are the functions of convolution and pooling layers in a CNN?

The **convolution layer** is responsible for extracting important features from an input image using small filters called kernels. These filters move across the image and detect patterns such as edges, corners, textures, and shapes. Deeper convolution layers can identify more complex features and objects. The **pooling layer** reduces the spatial dimensions of the feature maps while preserving important information. Common pooling methods include max pooling and average pooling. Pooling reduces computational complexity, memory usage, and overfitting while making the CNN more robust to small changes in object position.

3. Differentiate between image classification and object detection.

Image classification is a computer vision task that identifies the main object or category present in an entire image. It assigns one or more class labels, such as cat, dog, car, or bird, to the image. However, it does not identify the object's exact location. **Object detection**, on the other hand, identifies both the type and location of objects within an image. It can detect multiple objects and represents their locations using bounding boxes. Therefore, classification answers “**What is in the image?**”, while object detection answers “**What is present and where is it located?**”

4. Why is image preprocessing important before training a CNN model?

Image preprocessing is important because raw images may have different sizes, resolutions, brightness levels, orientations, and noise. CNN models require images to be presented in a consistent format for effective training. Common preprocessing techniques include resizing, normalization, noise removal, and data augmentation. Data augmentation may involve rotating, flipping, cropping, or zooming images to increase the diversity of training data. Proper preprocessing improves training stability, reduces unnecessary variations, prevents overfitting, and helps the CNN generalize better to new images. Therefore, preprocessing can significantly improve the accuracy and reliability of a CNN model.

5. What are the advantages of using benchmark datasets such as CIFAR-10 or Kaggle image datasets?

Benchmark datasets such as **CIFAR-10** and image datasets available on Kaggle provide standardized and often labeled data for developing and evaluating CNN models. They reduce the time and effort required to manually collect and organize large numbers of images. CIFAR-10, for example, contains images belonging to ten different object categories and is widely used for

image classification experiments. Benchmark datasets also allow researchers and students to compare different CNN architectures under similar conditions. They are useful for learning, experimentation, performance evaluation, model comparison, and testing new deep learning techniques in a consistent and reproducible manner.

AIM

To design, implement, and evaluate a Convolutional Neural Network (CNN) for image classification using a benchmark image dataset.

SOFTWARE REQUIREMENTS

- Python
- TensorFlow/Keras or PyTorch
- Google Colab
- Matplotlib
- Kaggle Dataset (e.g., CIFAR-10, Cats vs Dogs, Intel Image Classification)

OBJECTIVES

Students will be able to:

- Understand the architecture of Convolutional Neural Networks.
- Implement a CNN model for image classification.
- Train and evaluate a CNN using a real-world image dataset.
- Visualize training and validation performance.
- Analyze the effectiveness of CNNs for image recognition tasks.

PROCEDURE

PART A: DATASET PREPARATION

- Perform the following tasks:
- Download an image dataset from Kaggle or use CIFAR-10.
- Load the dataset.
- Preprocess images by resizing and normalization.
- Split the dataset into training, validation, and testing sets.

PART B: CNN MODEL IMPLEMENTATION

Build a CNN model consisting of:

- Convolution layers
- ReLU activation
- Max Pooling layers
- Fully Connected layers
- Softmax output layer

Compile the model using:

- Appropriate optimizer
- Cross-entropy loss function

PART C: MODEL TRAINING AND EVALUATION

Train the CNN model.

Record:

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss

- Testing accuracy

Plot:

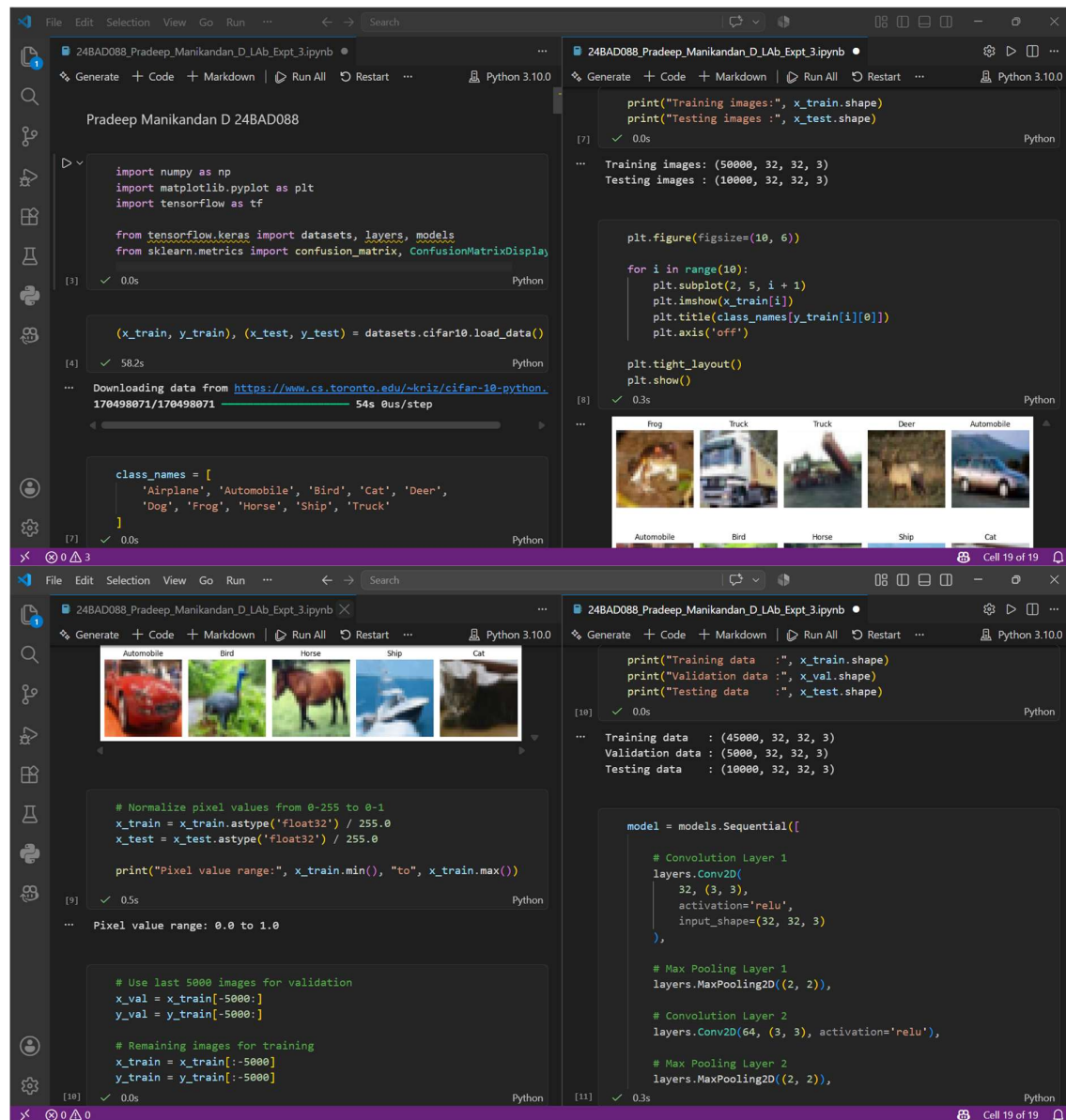
- Accuracy vs Epoch
- Loss vs Epoch

PART D: PERFORMANCE ANALYSIS

Analyze:

- Classification accuracy
- Confusion Matrix
- Sample predictions
- Misclassified images

Discuss the strengths and limitations of CNNs for image classification.



```
File Edit Selection View Go Run ... Search
24BAD088_Pradeep_Manikandan_D_Lab_Expt_3.ipynb
Generate + Code + Markdown | Run All | Restart ... Python 3.10.0

# Convolution Layer 2
layers.Conv2D(64, (3, 3), activation='relu'),

# Max Pooling Layer 2
layers.MaxPooling2D((2, 2)),

# Convolution Layer 3
layers.Conv2D(64, (3, 3), activation='relu'),

# Flatten
layers.Flatten(),

# Fully Connected Layer
layers.Dense(64, activation='relu'),

# Output Layer
layers.Dense(10, activation='softmax')

model.summary()

[11] ✓ 0.3s Python

c:\Users\PRADDEEP_MANIKANDAN\AppData\Local\Programs\Python\Python310\l
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

model.summary()

[12] ✓ 0.0s Python

Model: "sequential"

Layer (type) Output Shape Par
conv2d (Conv2D) (None, 36, 36, 32)
max_pooling2d (MaxPooling2D) (None, 15, 15, 32)
conv2d_1 (Conv2D) (None, 13, 13, 64) 18
max_pooling2d_1 (MaxPooling2D) (None, 6, 6, 64)
conv2d_2 (Conv2D) (None, 4, 4, 64) 36
flatten (Flatten) (None, 1024)
dense (Dense) (None, 64) 65
dense_1 (Dense) (None, 10)

Total params: 122,578 (478.79 KB)
```

```
File Edit Selection View Go Run ... Search
24BAD088_Pradeep_Manikandan_D_Lab_Expt_3.ipynb
Generate + Code + Markdown | Run All | Restart ... Python 3.10.0

Trainable params: 122,578 (478.79 KB)

Non-trainable params: 0 (0.00 B)

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("Model compiled successfully.")

[13] ✓ 0.0s Python

WARNING:tensorflow:TensorFlow GPU support is not available on native W!
Model compiled successfully.

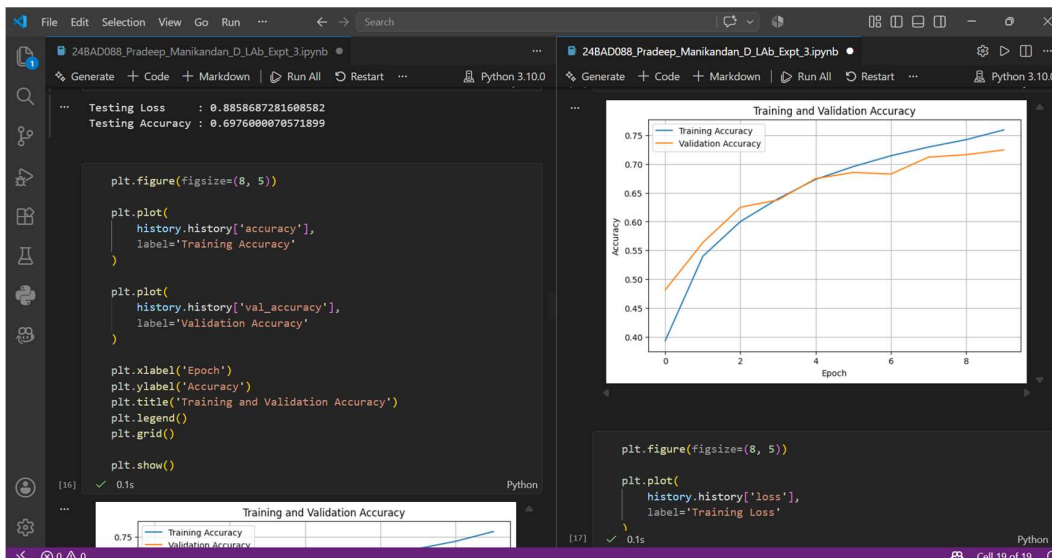
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(x_val, y_val)

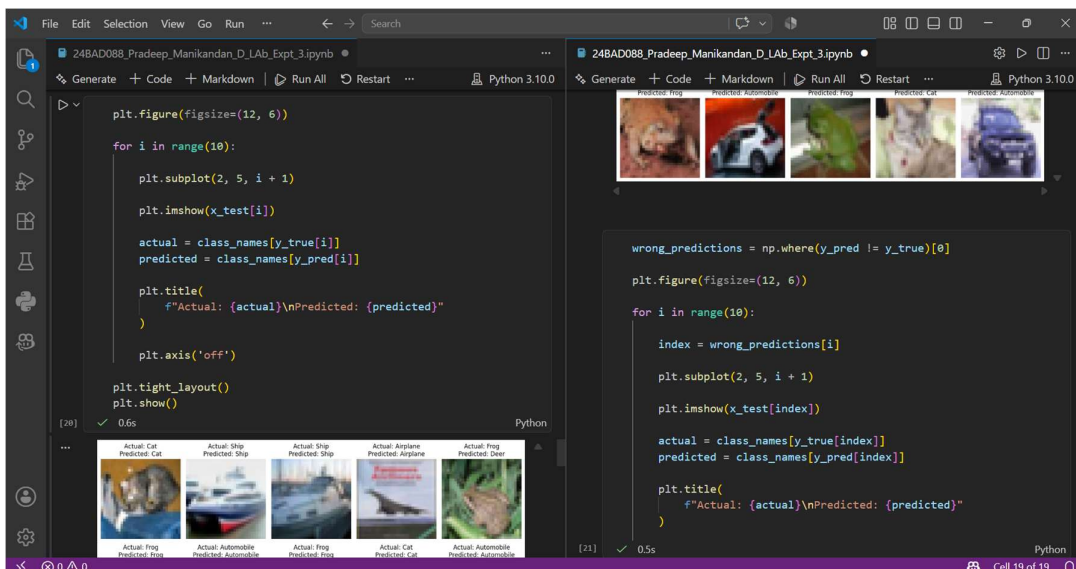
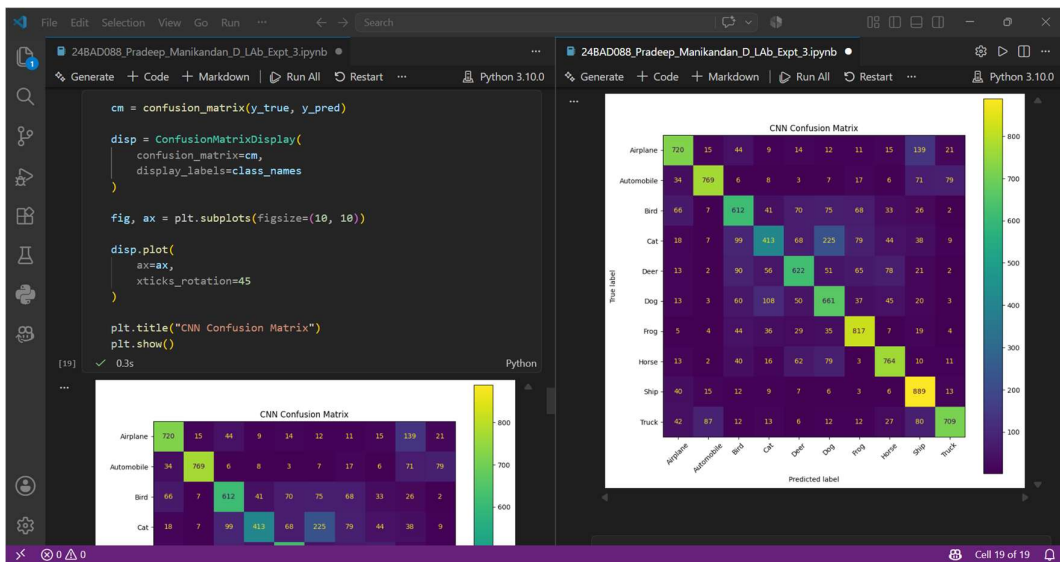
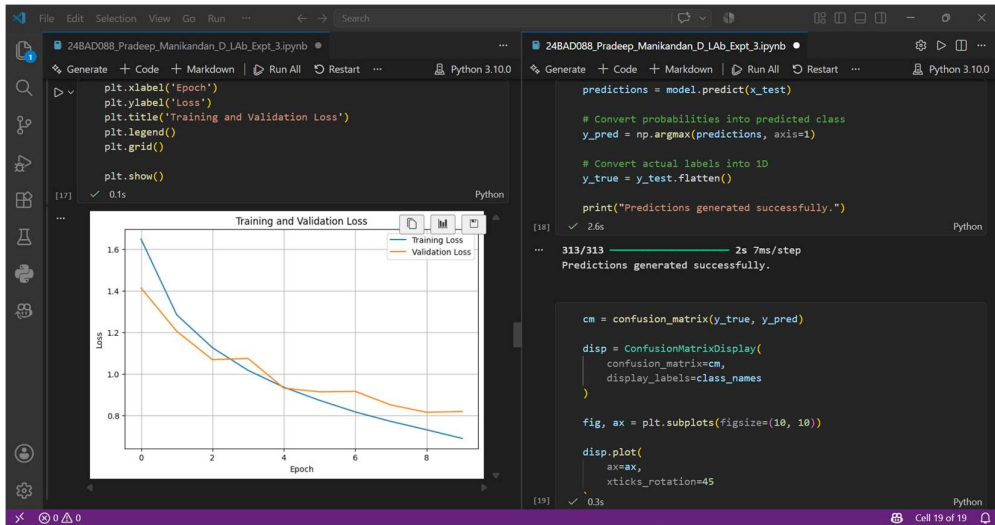
[14] ✓ 3m 6.2s Python

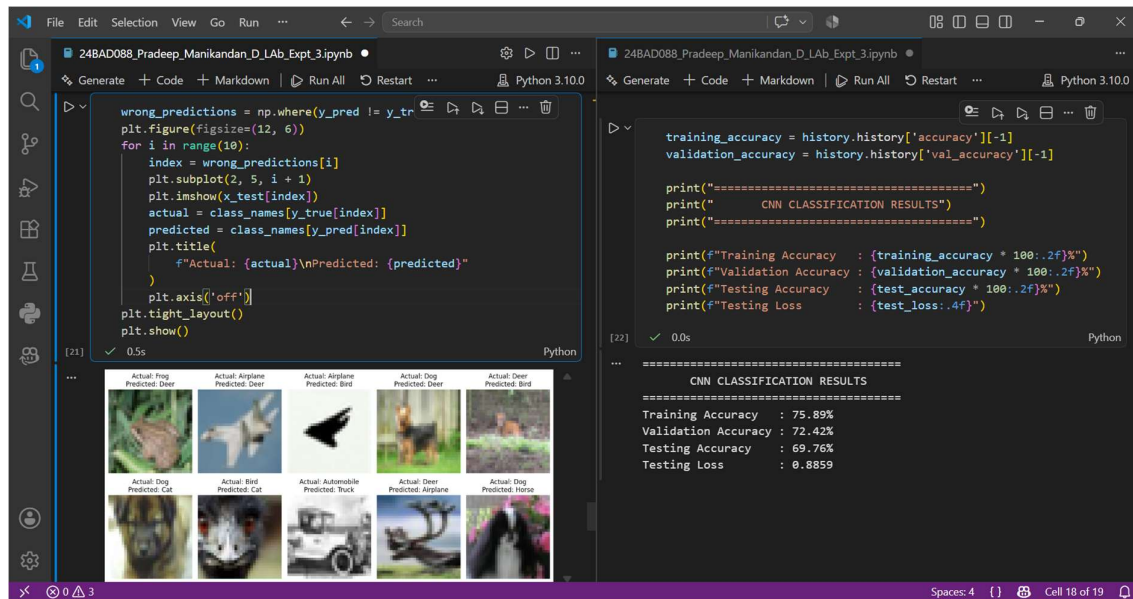
Epoch 1/10
704/704 15s 20ms/step - accuracy: 0.3935 - loss: 1
Epoch 2/10
704/704 14s 20ms/step - accuracy: 0.5399 - loss: 1
Epoch 3/10
704/704 24s 33ms/step - accuracy: 0.6002 - loss: 1
Epoch 4/10
704/704 26s 37ms/step - accuracy: 0.6396 - loss: 1
Epoch 5/10
704/704 25s 35ms/step - accuracy: 0.6732 - loss: 0
Epoch 6/10
704/704 29s 18ms/step - accuracy: 0.6954 - loss: 0
Epoch 7/10
704/704 13s 18ms/step - accuracy: 0.7141 - loss: 0
Epoch 8/10
704/704 13s 19ms/step - accuracy: 0.7292 - loss: 0
Epoch 9/10
704/704 13s 19ms/step - accuracy: 0.7420 - loss: 0
Epoch 10/10
704/704 13s 18ms/step - accuracy: 0.7589 - loss: 0

test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0

[15] ✓ 1.6s Python
```







EXPECTED OUTCOME

Students will gain practical experience in designing, training, and evaluating CNN models for image classification and understand how CNN architecture influences classification performance.

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. Why are convolution layers more effective than fully connected layers for image processing?

Convolution layers are more effective because they preserve the spatial relationships between pixels and automatically extract important features such as edges, textures, and shapes. They use small filters that scan across the image instead of connecting every neuron to every input pixel. This significantly reduces the number of parameters compared with fully connected layers. Convolution layers also use weight sharing, where the same filter is applied to different regions of an image. This makes CNNs computationally efficient and allows them to recognize the same feature even when it appears at different positions.

2. How does Max Pooling improve CNN performance?

Max Pooling reduces the spatial dimensions of feature maps by selecting the maximum value from a small region, commonly a 2×2 window. It retains the strongest and most important features while removing less useful information. This reduces the amount of computation required in subsequent layers and decreases memory usage. Max pooling can also help reduce overfitting because the network processes fewer values. Additionally, it makes the CNN less sensitive to small changes in the position of objects within an image, improving the model's ability to recognize features despite slight translations or distortions.

3. Which hyperparameters significantly affected your CNN model performance?

Several hyperparameters significantly affect the performance of a CNN model. The learning rate determines how quickly the model updates its weights during training. The batch size controls the number of images processed before updating the model parameters. The number of

convolution filters and kernel size influence the type and amount of features extracted from images. The number of epochs determines how many times the complete training dataset is processed. Other important hyperparameters include the optimizer, dropout rate, number of convolution layers, and pooling size. Proper tuning of these parameters can improve classification accuracy.

4. What challenges did you encounter while training the CNN model?

Some common challenges encountered while training a CNN model include overfitting, long training time, high computational requirements, and difficulty in selecting suitable hyperparameters. Overfitting occurs when the model performs well on training data but poorly on unseen testing data. Insufficient or imbalanced training images can also reduce model performance. Selecting an appropriate learning rate, batch size, number of epochs, and network architecture may require several experiments. CNN training can also require significant memory and processing power, especially for large image datasets. Proper preprocessing and regularization techniques help overcome these challenges.

5. Suggest two improvements that can increase the classification accuracy of your CNN.

Two effective improvements are data augmentation and hyperparameter tuning. Data augmentation creates additional variations of training images through operations such as rotation, flipping, cropping, and zooming. This increases data diversity and helps reduce overfitting, allowing the CNN to generalize better to unseen images. Hyperparameter tuning involves finding suitable values for parameters such as learning rate, batch size, number of filters, dropout rate, and number of epochs. Carefully tuning these parameters can improve the learning process and classification performance. Techniques such as batch normalization or transfer learning can further improve accuracy.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		